Applying Uncertainty Awareness in Deep Learning

KhineHninAye

June 2026

Abstract

Bayesian Neural Networks (BNNs) provide the standard framework for qualifying predictive uncertainty, which is one of the critical requirement for developing machine learning and deep learning models in high-stake domains such as autonomous driving, robotics and medical diagnostics. Despite BNN's theoretical achievements, BNN still remain absent in most modern machine learning and deep learning models due to high computational costs [1] which raise from requirement to preform Bayesian treatment over model's each individual parameters. This paper proposes a computationally efficient Bayesian model. Specifically we introduce a modified Bayesian architecture that applies Bayesian treatment per layer instead of per parameter. Therefore, reducing the interface overhead while serving the model capability for uncertainty-aware predictions. We test this approach with a Long Short-Term Memory (LSTM) framework for the task of weekly network traffic forecasting. Moreover, the primary results demonstrate the proposed layer-wise Bayesian achieves significant gains in training efficiency with less computational costs, indicating a pathway toward scalable uncertainty aware in future machine learning and deep learning systems.

1 Introduction

In high-stake domains such as healthcare, criminal justice, finance and autonomous vehicles, AI systems are highly consumed, where AI systems, make decisions in those areas such as autonomous driving in accident scenarios, unexpected roadside incidents to ensure safety and accountability to human life and in medical situations, where AI explains diagnoses or therapy recommendations in medical systems [2]. In this kind of high-stake domains where human life is on the line, model's uncertainty awareness is extremely important since a model being "accurate" most of the time is not enough anymore. Another important question should be asked is "Does the model know how reliable its predictions are?". For instance, in an autonomous driving scenario, a self-driving car without uncertainty awareness is driving at night. The AI camera sees something blurry ahead on the way, where AI predicts as [Object: PlasticBag, Confidence: 95%] so the car continues driving. But the object was actually a child wearing dark clothes, which result in an accident. In this kind of scenario, if only the model has uncertainty awareness, AI could predict [Object: PlasticBag, Confidence:70%, Uncertainty: High]. So the car recognizes that it is not certain about its prediction, so the system decides to slow down, use the radar (or) LiDAR to confirm the "Object" and prepare emergency braking. As the result, the vehicle avoids a dangerous decision.

Bayesian Neural Networks (BNNs) provide a standard framework for qualifying predictive uncertainty which is one of the critical requirement for developing machine learning and deep learning models in high-stake domains. Yet, they are not fully adopted and mostly forbidden in many modern machine learning and deep learning due to high computational costs since applying Bayesian treatment over model's each individual parameters. Jospin et al. [1] also stated that "In fact, most of the BNNs used in practice rely on methods that are approximately or implicitly Bayesian (Section V-E) since the exact algorithms are computationally too expensive"

In this paper, we present an architecture which we call "Per-Layer Bayesian inference" in which we apply Bayesian inference over each layers of the model instead of applying to each parameter which already theoretically reduce the computational cost. Alongside with this theory, we will apply this customized BNN architecture to the Long Short Term Memory (LSTM) which predict next upcoming week's network by using past 4 weeks network traffic pattern. This project proves that our architecture can apply Bayesian knowledge implicitly to the model while having low computational cost and compared to traditional fully implemented Bayesian inference.

1.1 Background and Motivation

While reviewing the work on Deep Knowledge Tracing by Piech et al. (2015) [3], a question raised. "What could possibly upgrade this paper and make it better". After further exploration in arXiv about Knowledge Tracing, Cheng et al. (2025) proposed an Uncertainty-Aware Knowledge Tracing (UKT) [4] model which is an explicit upgraded version of DKT. After reviewing the paper, the importance of the uncertainty in AI models has caught the attention, which led to observe that Bayesian Neural Network is one of most used approaches when applying uncertainty parameters to a model. Furthermore, while examining the "Hands-on Bayesian Neural Networks" tutorial by Jospin et al. [5], a particular sentence "In fact, most of the BNNs used in practice rely on methods that are approximately or implicitly Bayesian (Section V-E) since the exact algorithms are computationally too expensive" caught my attention.

Concurrently, numerous news reports highlighted instances where confident AI predictions resulted in significant errors, with substantial associated damage costs, led me realize that consequences will get worse when human lives are on the line. This underlying the importance of applying uncertainty into AI models deployed in high stake domains such as autonomous driving or medical diagnosis. This reasoning led me to recognize that implementing full Bayesian algorithm to a model is too computationally expensive that many AI/ML/DL models don't consider this. Furthermore investigating the underlying cause, the discovered statement is that in vanilla BNNs, Bayesian Interface is applied to all parameters of a model, moreover resulting in substantial computational expense. Furthermore, considering whether applying Bayesian Interface at per-layer level instead of per parameter level, might reduce computational costs while serving the benefits of uncertainty qualifications in a model.

1.2 Related Work

Cheng, W., Du, H., Li, C., Ni, E., Tan, L., Xu, T., and Ni, Y. (2025). Uncertainty-aware Knowledge Tracing. [4]
This paper proposes the importance of uncertainty awareness in the field of Knowledge Tracing and applied in Student Knowledge Tracing. This is one of the papers where it highlight the importance of uncertainty in ML/DL models of a specific field.

Jospin, L.V., Laga, H., Boussaid, F., Buntine, W. and Bennamoun, M. (2022). Hands-On Bayesian Neural Networks—A Tutorial for Deep Learning Users. [5]
This paper explains how to apply Bayesian statistics to neural networks to qualify prediction uncertainty. In this paper, they proposed why BNNs matter, what BNNs are, the standard Bayesian framework and inference methods. This is one of most hand-on guide for researchers and engineers who want to start applying uncertainty qualifications into their models.

Doan, B.G., Shamsi, A., Guo, X.-Y., Mohammadi, A., Alinejad-Rokny, H., Sejdinovic, D., Teney, D., Ranasinghe, D.C. and Abbasnejad, E. (2024). Bayesian Low-Rank LeArning (Bella): A Practical Approach to Bayesian Neural Networks. [6]
Tradational Bayesian Neural Networks assume that every weight in a neural network is a probability distribution. Which indicate that every weight has mean (μ) and variance (σ). For a neural network with 1 million weights, after applying Bayesian inference, we will get 1 million μ and 1 million σ which is 2 million learnable parameters. This paper recognizes if a model really requires uncertainty for every single weight. As the solution, they approximate the uncertainty using a low-rank representation. This models uncertainty in a compressed latent space which reduces the computational costs.

MedBayes-Lite: Bayesian Uncertainty Quantification for Safe Clinical Decision Support. [7]
This paper states that modern transformer models (BERT, ClinicalBERT, BioBERT, etc.) often make predictions with very high confidence, even when they are wrong. From the medical perspective, those situations are very dangerous since human lives are on the line. This paper recognize whether they can estimate how uncertain the model actually is without retraining the entire transformer. The proposed ideas was instead of converting the whole transformer into a Computationally expensive Bayesian Network, they introduces applying the lightweight Bayesian on the trained model which they call MedBayes-Lite.

Jantre, S., Bhattacharya, S. and Maiti, T. (2021) Layer Adaptive Node Selection in Bayesian Neural Networks: Statistical Guarantees and Implementation Details [8]
This paper states the massive size of DL models, containing unnecessary parameters, where extra neurons increase computation, increase memory usage and slow inference. The authors argue that removing edges does not necessarily reduce the actual network structure. They proposed a Bayesian

prior called spike-and-slap Gaussian prior. Their proposed idea was instead of removing connections between neurons, they remove entire neurons (nodes) which the neuron's the probability of being important for the model is low.

Current existing proposed uncertainty aware Bayesian inferences have indicated the importance of uncertainty implementation in improving the model reliability and computational cost reduction. The papers proposed alternative inference strategies and methods such as low-rank Bayesian learning and adaptive Bayesian architectures. However, applying Bayesian inference to layer-wise inference has been unexplored. In this work, we will investigate the integration of layer-wise Bayesian inference to the model to explore whether our proposed method can be achieved in reducing computational overhead.

1.3 Probability, Confidence and Uncertainty

This following session provide a brief view of the differences between Probability, Confidence and Uncertainty to improve the comprehensive understanding of subsequent chapters.

Class	Model A	Model B
Cat	0.95	0.55
Dog	0.05	0.45

Table 1: Probability predictions from two models.

Probability is a metathetical measurement of likelyhood of a variable or an event, score between 0 and 1 where 0 stands for "no possibility" and 1 is used when an event is absolutely certain to happen. For instance, As shown in Table 1, Model-A's output was ["cat": 0.95 ,"dog": 0.05], which indicates 95% probability of the given image being a cat [P(cat) = 0.95] and [P(dog) = 0.05]. On the other hand, counting on probability is not always correct nor mean model is correct. A model can output wrong answer with high probability whether it is because of insufficient training stages or inputs.

Confidence on the other hand, is a calibrated metric which is how strongly a model believe its own predictions. As shown in Table 1, Model-A is more confident then Model-B which can be seen that the Model-A's prediction results being decisive while Model-B's prediction results are wavering. In general, confidence is measured by simple math $confidence = max(P(y|x))$. But a model can be highly confident and completely wrong. For instance, if we show a dog picture to a model trained on cat and horse pictures, and never seen a dog, the softmax function will force the numbers, therefore it might output [horse: 99%, cat: 1%] with huge confidence even though the correct answer is neither.

Uncertainty is the amount of doubt that the model has on its own predictions. Uncertainty happens due to lack of complete knowledge or understanding over a topic. In some instances, a model can be in both high confidence and high uncertainty simultaneously, which indicates that the model believe over a specific variable, with still have no idea what is that. For an instance, a self driving car sees something blurry ahead. The car's camera is damaged, alongside with poor lighting and object is not one of the trained objects in model training data. Therefore, the model will not know the situation well. In this kind of instance, the model may output from what it learns from training data but the uncertainty is high. In many instances, it is about not knowing exactly what outcome will actually be, not the number itself. It is not probability either. As for another simple instance, when we are flipping a coin, it will be with 100% uncertainty while still have 50/50 possibility.

In the field of Artificial Intelligence, uncertainty is generally divided into 2 main categories. Understanding this will help engineers and researchers to understand what their model need. **Aleatoric Uncertainty**, also known as Data Uncertainty is caused by data noise, randomness or overlapping in training data. For instance, a autonomous driving car predicting the blurry thing in front of the car, whether due to heavy rain or blurry camera captures, poor lighting or messy data. On the other hand, **Epistemic Uncertainty**, also known as Model Uncertainty is caused when the model lack the knowledge over a situation. This indicates that the model hasn't see enough examples of a specific scenario when trained. Therefore, it's predictions in that specific area are highly uncertain. For instance, in a medical diagnostic model might have high epistemic uncertainty when facing a vary rare disease it was barely or never trained on.

2 Problem Statement

Bayesian Neural Networks (BNN) are one of most common types of Neural Networks used when applying uncertainty inference into AI models. This allows models to obtain the uncertainty of a predicted variable or an event of a scenario, which are highly important in high-stake domains such as medical diagnosis and autonomous driving. Although Bayesian Neural Networks (BNN) are widely recognized in the topic of uncertainty in AI models, yet, still not applied in many situations due to their high computational costs. The main underlying reason is that, Bayesian Inference is applied to all parameters (weight) of the model. The standard equation for Bayesian Inference is

$$W = \mu + \sigma \odot \epsilon$$

where W stands for Weight used during the LSTM forward pass, (μ) is the average or most likely value of the weight, (σ) is how uncertain the model is about the weight, and finally (ϵ) is a random number drawn from a normal distribution $\epsilon \sim \mathcal{N}(0, 1)$ In other words, applying mean (μ) and variance (σ) to all parameters of the model. Therefore, it means, for a 1 million parameter model, after applying standard Bayesian Inference, we will get a 1 million (μ) and 1 million (σ). Many researchers and engineers applied Bayesian Inference implicitly to their models (See in Related Work). However, none of them introduced the idea of per-layer Bayesian Inference. Put simply, it is about sharing one (μ) and one (σ) for each layers instead of each W. Therefore, in this paper, we will explore on how this per-layer Bayesian Inference works, how effective it is when combined with a model and finally as the main goal, how much compute power it reduced compared to a full Standard Bayesian Inference.

3 Method

Rather than discussing over theoretical proposition, proving and providing the method by a project is crucial for understandability and usability. Here, we integrated 3 Long-short Term Memory (LSTM) models which are applied on timeseries network forecasting project that use past 4 weeks of network traffic pattern and predict the upcoming next week's. Moreover, rather than using external ML frameworks such as TensorFlow or PyTorch, we will be developing all 3 models from scratch using numpy as standard library, and support libraries (such as time, os, pandas, etc...) for project efficiency. This will help us gain deeper insight. Therefore, will help us develop comprehensive understanding of how our per-layer Bayesian works under the hood. The first model is going to be a vanilla LSTM model without Bayesian, which we will be calling "LSTM" through out the paper. Followed by the second LSTM model with standard Bayesian Inference with standard $W = \mu + \sigma \odot \epsilon$ in the name of "LSTM-BNN". Moreover, we will be developing the third model which we call "LSTM-PLBNN" which is the proposed per-layer Bayesian model. Therefore, we will evaluate and determine the compute power and efficiency of each models to see whether our proposed model LSTM-PLBNN can preform with low computational costs while implicitly applying Bayesian Inference over the model.

3.1 Dataset

These models have specific key component requirements that many existing datasets does not meet. To overcome this problem, we produced a representative and realistic network traffic pattern dataset over 4 weeks, which also introduces certain features to match the realism, while supporting the project objectives along the way. The realistic patterns include

- Traffic changes depending on the time of the day such as business hours and night time
- 3 separated traffic ranges, starting from Peak Range, Normal Range to Quiet Range based on time of the day.
- Weekend Traffic was Reduced to reflect on reality, some traffic in early morning to introduce the work of dogwatches and some checkup works.
- Generated each data for every 30 min, in the total of 48 data per day
- Simulated 4 weeks of the data, sufficient to train the model
  Exported to a csv for analysis and evaluation and ease of use (timestamp, traffic)

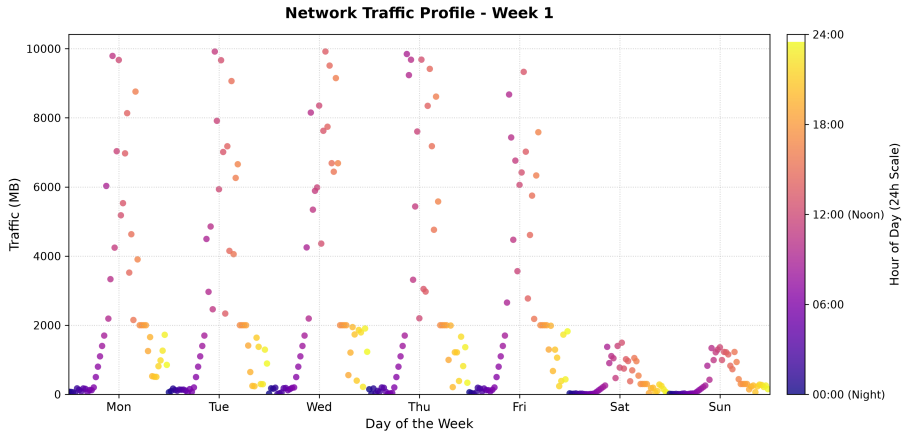


Figure 1: Week-1's Network Traffic Pattern

3.2 Timestamp Management

To support model's understanding and accessibility over the timestamps, such as 12AM and 1PM are proximally close to each other, we incorporate feature engineering. Providing models with raw timestamps such as 2026-07-22:30:00 will introduce complexity. The underlying issue is that if we assign raw hours, the model may assume that 12AM and 1PM is distant, in which reality is only one hour apart. For instance, if we assign "0" for 12AM and "23" for 1PM, the model will treat them as standard numerical values where 0 is numerically distant from 23 which in this situation is incorrect. The fundamental principle is that time is a circle and not a line.

To address this problem, Trigonometry such as Sine and Cosine were adopted for feature engineering of this project, therefore applied to all our models. Instead of providing raw hour data into model, we are going to convert the hour into 2 Dimensional circular representation using sin and cos trigonometry functions, presented in equation [1] and [2]

$$\text{hour}_{\sin} = \sin\left(\frac{2\pi h}{24}\right) \tag{1}$$

$$\text{hour}_{\cos} = \cos\left(\frac{2\pi h}{24}\right) \tag{2}$$

As stated in equation [1] and [2], we can assign the 11PM (23h) and 12AM (0h) as 0h is sin(0) = 0, cos(0) = 1, therefore, can be described in the format of ($\text{hour}_{\sin}$, $\text{hour}_{\sin}$) which corresponds to (0,1) where 23h is

$$\sin\left(\frac{23}{24}2\pi\right) \approx -0.259, \cos\left(\frac{23}{24}2\pi\right) \approx 0.966$$

The final result of 11PM(23h) was (-0.259, 0.966) which are close to (0,1). Therefore, the model learns that 11PM is near midnight and not distant.

3.3 LSTM Model

The traditional Recurrent Neural Network (RNN) was previously adopted for this project. However, due to gradient exploding and vanishing gradients problems after testing phase, we decided to transaction to the updated variant of RNN, named Long Short-Term Memory (LSTM) which introduces forget gates, cell gates and other additional mechanisms to address the gradient exploding and vanishing problems from standard RNN. All models of this paper adopt the same Standard LSTM model architecture, therefore LSTM-BNN and LSTM-PLBNN adopt the identical standard LSTM cell architecture. In LSTM-BNN and LSTM-PLBNN, the only modification are that the weights and bias calculations are replaced with parameters sampled from learned Gaussian Posterior distributions which are obtained via Bayesian Inference.

Data Preparation stage was also adopted for all 3 models of the paper. Upon loading the data from csv (which contains timestamp, traffic_mb), the traffic values were normalize using equation [3]

$$x_{\text{norm}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \tag{3}$$

where x represents original traffic value, $x_{\min}$ is the minimal value and $x_{\max}$ is the maximum value in the dataset and finally, x_{norm} is the final normalized value. For instance, if we have $x_{\min}$ = 100, $x_{\max}$ = 500, for the value of 200MB, we can calculate as

$$x_{\text{norm}} = \frac{200 - 100}{500 - 100} = \frac{100}{400} = 0.25$$

where the results are stated in between 0 and 1, which normalization prevents from larger scales dominating the model training and smaller scales vanishing. Consequently, the vector of $x_(\text{t})$ is defined as

$$x_t = \begin{bmatrix} \text{traffic}_{\text{norm}} \\ \text{hours}_{\text{sin}} \\ \text{hours}_{\text{cos}} \\ \text{is\_weekend} \end{bmatrix} \tag{4}$$

Therefore, the normalized data is now prepared for model training for model to learn and access for further prediction tasks.

To establish the foundational understanding of LSTM networks, readers are strongly advised to first consume the relevant LSTM articles such as wikipidia [[9]] since the paper adopts standard LSTM equations and architectures.

In the input concatenation, at each timestep t, the $h_{\text{t-1}}$ and current input x are concatenated into a single vector z_{t}, resulting [5]

$$z_t = \begin{bmatrix} h_{t-1} \\ x_t \end{bmatrix} \tag{5}$$

Forget Gate determines how much previous memory should be trained. This function outputs numbers between 0 and 1 because of the sigmoid function, where 0 stands for Forget completely and 1 stands for keeping the memory with element-wise multiplication, stated in equation [6]

$$f_t = \sigma(W_f z_t + b_f) \tag{6}$$

Input Gate controls how much new information and value should be inserted into memory, therefore determines how much new information is allowed into memory, stated in equation [7]

$$i_t = \sigma(W_i z_t + b_i) \tag{7}$$

Candidate Cell/Memory develop and creates candidate information which could be stored, stated in equation [8]

$$\tilde{C}_t = \tanh(W_c z_t + b_c) \tag{8}$$

Cell State combines and compute the old memory with new memory. The new cell state is created by combining previous memory after forgetting and new candidate memory, stated in [9]

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \tag{9}$$

Output Gate controls how much information from the cell state C_t is exposed to the hidden state, therefore, LSTM does not directly output its entire memory and instead, it uses this gate to select and process useful information, stated in h_t[10]

$$o_t = \sigma(W_o z_t + b_o) \tag{10}$$

Hidden State is the output representation of LSTM at time step t. This parameter is passed to the next LSTM time step and eventually, to the final prediction layer. C_t is the cell state, which is the LSTM s long germ memory, where it stores information accumulated over previous time steps, stated in[11]

$$h_t = o_t \odot \tanh(C_t) \tag{11}$$

Output Layer converts the final hidden state representation to the predicted values. For sequence forecasting, timeseries forecasting final hidden state is written in h_T, contains all important values and information from the entire timeline sequence, stated in [12]

$$\hat{y} = W_y h_T + b_y \tag{12}$$

Loss Function measures how different the predicted values are from the true value, where y is ground truth value and $\hat{y}$ is predicted value and compute the error between predicted value and ground truth, stated in [13]

$$L = \frac{1}{2}(\hat{y} - y)^2 \tag{13}$$

Backpropagation Through Time (BPTT) is the algorithm used in RNNs and other variants of RNNs. This process enables the model to learn patterns and learn sequential and time series data by unrolling the network across time and update the weights alongside to minimize errors. Gradients move backward through previous time steps to learn the errors and update the current parameters.

Output Gradient calculates the gradient of the loss with repeatedly to the predictions, stated in [14]

$$\frac{\partial L}{\partial \hat{y}} = \hat{y} - y \tag{14}$$

Updating weight equation updates the model parameters after calculating their gradients, stated in [15]

$$W = W - \eta \frac{\partial L}{\partial W} \tag{15}$$

3.4 LSTM-BNN Model

Followed by second model which we named LSTM-BNN, we now transform every LSTM parameters into probability distribution to follow standard Bayesian Inference principles. Instead of learning a fixed weights adopted by first LSTM model, we now assume that the weight is uncertain and treat each weight as a random variable. For instance, instead of $W = 0.21$, we now learn with $W \sim \mathcal{N}(\mu, \sigma^2)$ where μ is expected value (mean) and σ is uncertainty (standard deviation). Therefore, every weight is represented as (μ, σ) rather than a single number.

Prior Distribution is a probability distribution which Bayesian learning assumes weights follow the prior distribution before seeing any data which we used in equation [16], where σ_{p} which the model assume every weight is near zero before training.

$$p(W) = \mathcal{N}(0, \sigma_p^2) \tag{16}$$

Variational Posterior is a simple mathematical distribution used to approximate an exact, complex posterior distribution when calculating directly is computationally near impossible. The true posterior in equation [17] cannot be computed exactly for neural networks. So we use Variational Posterior with the equation [18]. Therefore, instead of learning W, we learn (μ, ρ). We did not apply sigma directly as the variance must always satisfy $\rho > 0$ as gradient descent could make sigma negative. Therefore the model learns p instead. Then, we use softplus function [19] which guarantees $\rho > 0$.

$$P(W \mid D) = \frac{P(D \mid W)\, P(W)}{P(D)} \tag{17}$$

$$q(W) = \mathcal{N}(\mu, \sigma^2) \tag{18}$$

$$\sigma = \log(1 + e^{\rho}) \tag{19}$$

Reparameterization In this project, instead of sampling $W \sim \mathcal{N}(\mu, \sigma^2)$ directly, we sample first with $\epsilon \sim \mathcal{N}(0, 1)$. Therefore, constructs with $W = \mu + \sigma\epsilon$ since ϵ is independent of (μ, σ) which allows gradients to flow through backpropagation easier.

LSTM equations and LSTM equations remain exactly the same once the weights are sampled. For instance, we still use equation [6] for Forward Gate and so on. Every gates works in the same way.

KL Divergence is a Bayesian regularizer which measures how different the posterior is from the prior which we compute with equation [20]

$$KL = \sum \left[\log\left(\frac{\sigma_p}{\sigma}\right) + \frac{\sigma^2 + \mu^2}{2\sigma_p^2} - \frac{1}{2} \right] \tag{20}$$

In this equation, $\log\left(\frac{\sigma_p}{\sigma}\right)$ is used to penalizes extremely small or extremely large uncertainty. $\frac{\sigma^2}{2\sigma_p^2}$ keeps the learnt uncertainty close to the prior uncertainty. $\frac{\mu^2}{2\sigma_p^2}$ is used similarity to L2 weight decay, forcing the mean weights to near 0 unless the data strongly supports larger values. Finally, followed by $-\frac{1}{2}$ which is a constant directly from mathematical derivation of the KL divergence between 2 Gaussian distributions that ensures the expression evaluates to 0 when posterior and prior are identical.

Loss Function of a Standard LSTM [21] which minimizes only predictions error where our LSTM-BNN's Loss Function [22] minimizes both Prediction error (MSE) and KL divergence which the learned posterior weight distributions remain close to prior unless the data points the sufficient evidence to move away from it.

$$L = MSE \tag{21}$$

$$L = MSE + \frac{KL}{N} \tag{22}$$

Gradient Updates is the process of adjusting model's parameters using the gradient of its loss function. Since the model learns two sets of parameters which are μ (mean) which determines average weight value and ρ (Variance parameter) which controls uncertainty through $\sigma = \text{Softplus}(\rho)$. During backpropagation, gradients are computed separately for mu and rho. We also adds the gradient of the KL divergence before updating both parameters with gradient decent.

Summarization of the model which we named LSTM-BNN is a customized LSTM model developed for network traffic forecasting, which treats every parameters (weights) as probability distributions rather than a single fixed parameter value. Rather than storing a single fixed parameter, we assume that all weights are now represented with (mu, rho), therefore, transformed to sigma using the softplus function. During each forward pass, we sample new weights from Gaussian Posterior using reparameterization which enables network to account for uncertainty in their parameter. Training minimize the loss of 2 components (MSE and KL Divergence) which guide the learned posterior toward a Gaussian Prior Distribution. During inference, we preform multiple stochastic forward passes (Monte Carlo sampling) and generate series of predictions rather than a single deterministic output. Finally, the final prediction is computed as mean prediction alongside with with the prediction standard deviation and confidence intervals which comes alongside with measure of uncertainty. Although applying standard Bayesian Inference to LSTM model improves robustness and reliability, we still requires substantially more compute power and memory since each parameter has both mu and rho values and is sampled repeatly.

3.5 LSTM-PLBNN Model

Followed by the third model, which is our proposed model which we named LSTM-PLBNN which stands for Long Short-Term Memory - Per Layer Bayesian Neural Network. Although LSTM-PLBNN is not entirely a new model, this is a small optimized variant of LSTM-BNN which is designed and developed to reduce computational costs while supporting uncertainty estimation with implicit BNN Inference. This model introduces an architecture where each layer shares a single uncertainty parameter **rho** instead of heaving independent uncertainty for individual weight in each layer of the model.

Standard LSTM computation and equations are the identical for all models (LSTM, LSTM-BNN and LSTM-PLBNN) where we developed with the same equations such as Forget Gate [6], Input Gate[7], Candidate Cell [8], Cell State [9], Output Gate [10], Hidden State [11], Output Layer [12]. In the third LSTM-PLBNN model, the only value of W and b were different due to different Bayesian sampling.

Weight Representation is now modified. In the previous LSTM-BNN model, each individual weight has their own probability distribution. Rather than W_{ij} in standard LSTM model, LSTM-BNN model learns μ_{ij}, ρ_{ij}, therefore formalized with $\sigma_{ij} = \log(1 + e^{\rho_{ij}})$. Each individual weight is sampled independently with $W_{ij} = \mu_{ij} + \sigma_{ij}\epsilon_{ij}$ where $\epsilon_{ij} \sim \mathcal{N}(0, 1)$, which means all weight has their own uncertainty calculations in LSTM-BNN model.

For a weight matrix [23],

$$W = \begin{bmatrix} w_{11} & w_{12} \\ w_{13} & w_{14} \end{bmatrix} \tag{23}$$

is sampled to [24] in a standard LSTM-BNN model.

$$\begin{aligned} w_{11} &= \mu_{11} + \sigma_{11}\epsilon_{11} \\ w_{12} &= \mu_{12} + \sigma_{12}\epsilon_{12} \\ w_{13} &= \mu_{13} + \sigma_{13}\epsilon_{13} \\ w_{14} &= \mu_{14} + \sigma_{14}\epsilon_{14} \end{aligned} \tag{24}$$

In the proposed LSTM-PLBNN, we introduced a few custom changes where individual weight has its own mean μ but all weights in one parameter tensor share the same variance. So, instead of having ρ_{11},ρ_{12}, ρ_{13}, ρ_{14}, the proposed model only have ρ_{layer}. Therefore, $\sigma_{\text{layer}} = \log(1 + e^{\rho_{\text{layer}}})$. So the sampled matrix becomes $W = \mu + \sigma_{\text{layer}}\epsilon$ where $\varepsilon \sim \mathcal{N}(0, 1)$. This means one scaler shared across the entire matrix layer. Therefore, every single weights changes and update together by the same stochastic factor rather than independently.

KL Divergence regularization is another major difference in our LSTM-PLBNN model. Unlike LSTM-BNN's KL Divergence [20], which has one KL contribution per parameter element, our LSTM-PLBNN groups the variance term. Assuming N weights in tensor, we compute with [25]

$$KL = N\left(\log\frac{\sigma_p}{\sigma} + \frac{\sigma^2}{2\sigma_p^2} - \frac{1}{2}\right) + \frac{\sum\mu^2}{2\sigma_p^2} \tag{25}$$

where only means remain element-wise and uncertainty term is shared across the whole tensor.

Gradient Update, was also changed. In standard LSTM-BNN, for every individual weight [26] and [29] where individual parameter learns their own uncertainty.

$$\mu_{ij} \leftarrow \mu_{ij} - \eta\frac{\partial L}{\partial \mu_{ij}} \tag{26}$$

$$\rho_{ij} \leftarrow \rho_{ij} - \eta\frac{\partial L}{\partial \rho_{ij}} \tag{27}$$

But in our proposed LSTM-PLBNN model, means are still updated individually as shown in [28] but we now only compute a single gradient for the shared parameter (uncertainty) as shown in [29] with [30] with the chain rule of [31] where $\partial\sigma/\partial\rho = \text{sigmoid}(\rho)$.

$$\mu_{ij} \leftarrow \mu_{ij} - \eta\frac{\partial L}{\partial \mu_{ij}} \tag{28}$$

$$\rho_{\text{layer}} \leftarrow \rho_{\text{layer}} - \eta\frac{\partial L}{\partial \rho_{\text{layer}}} \tag{29}$$

$$d\sigma = \left(\sum_{i,j}\frac{\partial L}{\partial W_{ij}}\right)\epsilon, \tag{30}$$

$$\frac{\partial L}{\partial \rho} = \frac{\partial L}{\partial \sigma}\cdot\frac{\partial \sigma}{\partial \rho}, \tag{31}$$

In summary, the proposed LSTM-PLBNN is the another variant of LSTM-BNN, designed and developed to reduce computational cost while serving and supporting Bayesian uncertainty inference to the model. Instead of assigning ρ to every single weight and bias of the model, we now assigns one shared variance for each layer or parameter group of the model. Therefore, in weight sampling, we only use a single random variable for each parameter group and therefore, each weight matrix is computed using this uncertainty value. The KL divergence and gradient are customized to support the shared variance parameter architecture instead of many individual values. This reduces the number of learnable uncertainty parameters which lowers the computational costs such as decreases in FLOPs (Floating point operations) and training time, while enabling Monte Carlo Prediction to estimate uncertainty of the prediction. Moreover, the customized model is a trade off between and uncertainty estimations of full model while offering computational efficiency.

4 Results

Results of certain calculations may depend on the computational resources availability to the user. For this research, a standard laptop was used. Therefore, the results will vary accordingly. Prior to the results and discussions, this session outlines how the final results of the model were measured and analysis-ed. In the results, we will evaluate and analysis the RMSE, MAE, MAPE , prediction pattern accuracy, uncertainty quantification, FLOPs (Floating-Point Operations), memory usage and computational time requirements.

4.1 Result Calculation Methods

Execution Time (sec) is measured with the total amount of time required for the model to complete training and prediction. In the provided model code development, the timer starts right after the code runs and therefore stops after the prediction is finished, by using python's time library. Therefore, the difference between "end time" and "started time" is calculated as total execution time required for the model.

Peak Memory (bytes) are measured the maximum amount of RAM usage during the execution. The code is developed to make background monitoring thread which repeatably checks the process's RSS (Resident Set Size) using psutil library. Therefore, the highest RSS is recorded as peak memory bytes. Thus, this is the maximum memory required for the model to train and generate predictions. Lower memory usage indicates better computationally efficient model.

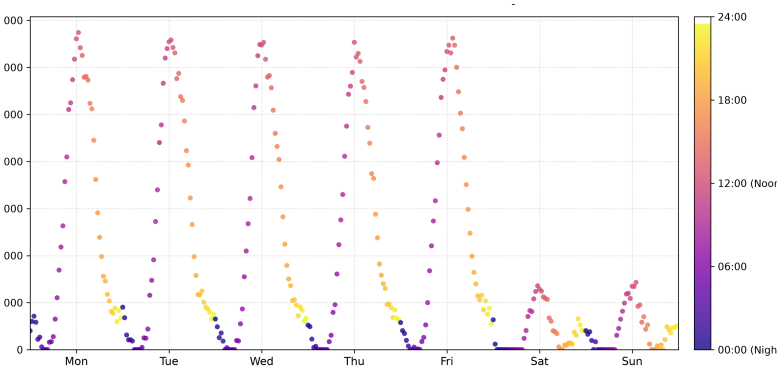
FLOPs (Floating-Points Operations) are the total number of Floating-Points Operations produced by the model during running time. FLOPs are used to measure the size of the model by counting the amount of math calculations required to process. For this paper's first model (LSTM), FLOPs are calculated in Matrix multiplications in LSTM Gates, Output Layer, Forward propagation, BPTT and Inference. Therefore, the final calculations is formalized in [32] where MAC (Multiply Accumulate Operation) as estimated as 2 flops since one for multiplication and one for addition.

$$\textbf{Total Flops} = 2 * (\textbf{Training MACs} + \textbf{Inference MACs}) \tag{32}$$

When FLOPs are calculated for Bayesian Models (LSTM-BNN, LSTM-PLBNN), estimated additional included computational overhead of Sampling Bayesian Weights, KL Divergence, Bayesian Gradient transformations and Monte Carlo Sampling. The LSTM-PLBNN model reduces computational overhead since uncertainty is represented per layer instead of per individual weights, which results in fewer sampling and KL computations.

RMSE, **MAE** and **MAPE** are all calculated by comparing mode's prediction with the "ground truth". MAE (Mean Absolute Error) is the the absolute difference between prediction and reality. RMSE (Root Mean Squared Error) gives more penalty to large errors since it squares every error. MAPE (Mean Absolute Percentage Errors) measures the errors as percentage of true value.

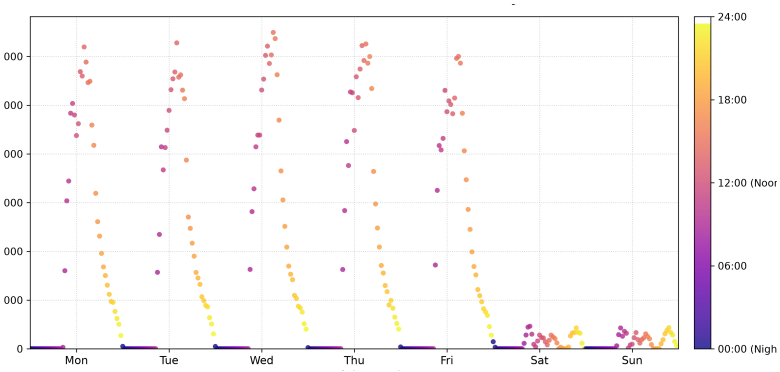
4.2 Final Results



LSTM

Duration	185.78 s
Peak Memory	75.32 MB
FLOPs	85.67B
RMSE	1321.52
MAE	777.02
MAPE	184.23%

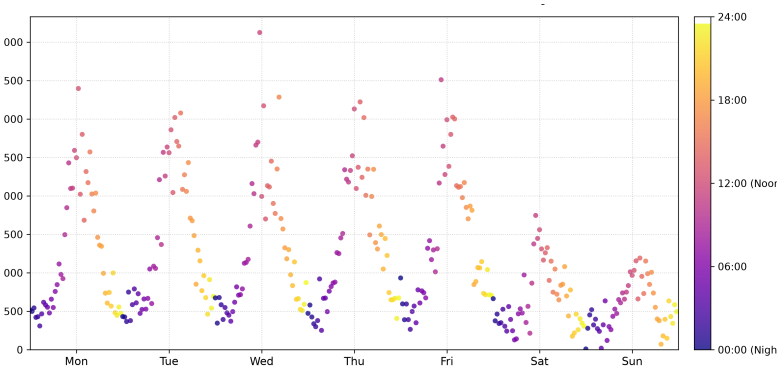
(a) LSTM Model Results



LSTM-BNN

Duration	718.07 s
Peak Memory	76.27 MB
FLOPs	93.35B
RMSE	1474.17
MAE	852.84
MAPE	76.65%
uncertainty	10.34%

(b) LSTM-BNN Model Results



LSTM-PLBNN

Duration	376.54 s
Peak Memory	76.01 MB
FLOPs	88.53B
RMSE	2067.73
MAE	1145.79
MAPE	402.41%
uncertainty	56.06%

(c) LSTM-PLBNN Model Results

Total 4 weeks of network traffic pattern data were used to train the models, therefore predict the subsequent week's (week 5) network traffic pattern. (see the training data pattern sample in [1]). As shown in [2a], LSTM model predicted the patterns correctly where the traffic during weekdays was high, while the patterns were low on weekends. Similarly, LSTM-BNN also predicted the week-5 pattern correctly, as shown in [2b]. However, in LSTM-PLBNN as shown in [2c], the predicted patterns were marginally acceptable as the patterns were a somewhat messy, yet some underlying patterns are discernible.

Model	**Time (s)**	**Memory (MB)**	**FLOPs**	**RMSE**	**MAE**	**MAPE (%)**	**Avg. Uncertainty (%)**
LSTM	185.78	75.32	85.67B	1321.52	777.02	184.23	–
LSTM + BNN	718.07	76.27	93.35B	1474.17	852.85	76.65	10.34
LSTM + PLBNN	376.54	76.02	88.53B	2067.73	1145.79	402.41	56.06

Table 2: Performance Comparison of the Proposed Models

[2] presents a performance comparism of all 3 models, which include Time (s), Memory (MB), FLOPs (Billions), RMSE, MAE, MAPE (%) and average uncertainty for models such as LSTM-BNN and LSTM-PLBNN.

In Table [3], we calculated and compared the differences between LSTM-PLBNN and LSTM-BNN model, by first, removing LSTM compute power to get BNN specific compute power. Therefore, we compared the pure BNN compute power between LSTM-BNN and LSTM-PLBNN models to determine whether our proposed model achieved the intended goal, reducing the computational power while serving uncertainty. We calculated the differences and reduction of the results, in both values and percentage.

Metric	Duration (s)	Peak Memory (MB)	FLOPs (Billions)
Combined Model Measurements			
LSTM–PLBNN	718.07	76.27	93.35
LSTM–BNN	376.54	–	88.53
Baseline LSTM	185.78	75.32	85.67
Isolated (Pure) BNN Computation			
Pure PLBNN	532.29	0.95	7.68
Pure Standard BNN	190.76	0.69*	2.86
Absolute Difference	**341.53**	**0.26**	**4.82**
Relative Difference (%)	**64.16%**	**27.37%**	**62.76%**

*Estimated value.

Table 3: Comparison of Pure BNN Computational Power Across Metrics

In the final results, the results indicate that our proposed model has reduced significant amount of computational overhead compared to standard BNN. However, improvement comes with a trade off where our proposed model's prediction is not as prefect as standard LSTM or LSTM-BNN model, yet it still uncertain about its predictions at approximately 56%, which indicate that uncertainty calculation is working as intended.

5 Discussion

In this paper, we introduced a new model named LSTM-PLBNN to solve the problem of computational overhead when the Standard Bayesian Inference is directly apply to the models. The final results of LSTM-PLBNN shows both trade-off and improvements in the model's performance.

5.1 Predictions Analysis

For trade-offs, as shown in figure [2c], LSTM-PLBNN only predicted near acceptable patterns which is caused by the fact that Bayesian uncertainty was applied in per layer instead of per parameter where all weights of a layer were forced to shares the same variance. For instance, if a weight is uncertain but another weight is certain, we still force the same variance on both since they are on same layer. A new approach will be required to fix this problem in the upcoming researches. But on the improvement side, while LSTM-PLBNN predicted pattern results were unstable, the model still can determine whether it is certain or uncertain on it's own predicted the results. As shown in table [2], the uncertainty is high as 56% where the model knows it's predicted results are uncertain, where it is actually unstable results. This indicate that LSTM-PLBNN model becomes uncertain when it's predictions were actually unstable which meet our requirements. In many real-word applications, for a model understanding that a prediction is unreliable and uncertain is as important as the prediction itself, where the uncertainty becomes additional part of information for decision-making. So the proposed model LSTM-PLBNN indicate higher uncertainty when its predictions are unstable, indicating that the model is capable of reflecting its own lack of confidence under challenging prediction conditions.

5.2 RMSE, MAE, MAPE Analysis

Since training data was generated to get the patterns with a small randomness in values, the values are not intended to extract exact fixed values but rather to determine the underlying patterns. For instance, in training, the data pattern might be $100 \rightarrow 150 \rightarrow 200$ but in model prediction, it could be $90 \rightarrow 145 \rightarrow 210$. Although the predicted values are different in numerically from ground truth, the model successfully captures overall pattern. Since RMSE MAE MAPE measures numerical closeness and does not represent whether the temporal traffic pattern behavior was captured. A model can follow the correct behavior but still have high error in this type of setup. So in this project, these metrics does not reflect whether the model has learned the underlying traffic pattern.

5.3 Computational Efficiency Analysis

In whole model comparisons, as shown in [3], the baseline LSTM has the fastest compute time since it doesn't need additional Bayesian Inference compute costs. Model 2 (LSTM-BNN) was increased

in significant computational costs as adding BNN increases the parameter compute. Similarly, the proposed model LSTM-PLBNN also have higher computational costs compared to standard LSTM but significantly reduces the cost when compared to standard BNN such as reducing . This indicates that as intended, the LSTM-PLBNN requires less FLOPs, finishes much faster and less computational overhead to network. However, the memory changes slightly, since all 3 models still need and occupies almost all memory. Peak memory measures the maximum RAM used at during execution time. The peak memory is determined by what is simultaneously in the model during a model's execution. This indicates that the proposed model improves and reduces computational requirements without increasing model's memory maximum requirements.

In summary, based on the final results, the proposed LSTM-PLBNN model address the main limitation of a standard Bayesian Neural Network which is their high computational overhead when applied to co-models. As benefits, the proposed LSTM-PLBNN successfully reduced computational overhead when it comes to FLOPs and execution time, while serving the uncertainty correctly. However, as a trade off, the proposed LSTM-PLBNN's predictions were a bit unstable compared to LSTM-BNN and LSTM models since the model's weights are forced to use one uncertainty parameter in each layer. This indicates that the current proposed model will need a bit more architecture changes and methods to make the predictions stable. Finally, this model is a variant of a Bayesian Neural Network, designed to reduce computational overhead, making this more practical and consideration where computational resources and execution time are important considerations.

6 Conclusion

This paper introduces Per-Layer Bayesian LSTM, applied uncertainty parameter per layer instead of per weight to reduce computational overhead while maintaining implicit Bayesian Inference. Trained on 4 weeks of network traffic data to predict upcoming week's. The proposed model successfully reduced significant amount of computational overhead (FLOPs and execution time) compared to standard Bayesian Architecture, successfully computed significant amount of uncertainty when it's own predicted pattern results were unstable, indicating that the model is capable of reflecting its own lack of confidence under challenging prediction conditions. For tradeoffs, the traffic pattern results were only near acceptable, caused by the architecture as shared layer-wise variance overlook weight level difference. This model requires architectural refinements to make the predictions stable. Future work should optimize the architecture for stable predictions with reliable uncertainty. These results are only validated on standard laptop with small dataset. For large scale testings such as computer vision, using capable hardware with larger datasets are recommended. Alternative designs such as grouping similar weights or applying uncertainty to weights in every other layer are also recommended to explore further.

References

[1] Jospin, L.V., Buntine, W., Boussaid, F., Laga, H. and Bennamoun, M. (2020). *Hands-on Bayesian Neural Networks – a Tutorial for Deep Learning Users*. arXiv:2007.06823 [cs, stat]. Available at: `https://arxiv.org/abs/2007.06823`.

[2] Kumar Yadav Student, R. (2025). *Explainable AI for High-Stakes Decision-Making Systems*. IJRTI, 10, p.378. Available at: `https://www.ijrti.org/papers/IJRTI2504265.pdf`.

[3] Piech, C., Spencer, J., Huang, J., Ganguli, S., Sahami, M., Guibas, L. and Sohl-Dickstein, J. (2015). *Deep Knowledge Tracing*. arXiv:1506.05908. Available at: `https://arxiv.org/pdf/1506.05908`.

[4] Cheng, W., Du, H., Li, C., Ni, E., Tan, L., Xu, T. and Ni, Y. (2025). *Uncertainty-aware Knowledge Tracing*. arXiv:2501.05415. Available at: `https://arxiv.org/abs/2501.05415`.

[5] Jospin, L.V., Laga, H., Boussaid, F., Buntine, W. and Bennamoun, M. (2022). *Hands-On Bayesian Neural Networks—A Tutorial for Deep Learning Users*. IEEE Computational Intelligence Magazine, 17(2), pp.29–48. Available at: `https://doi.org/10.1109/mci.2022.3155327`.

[6] Doan, B.G., Shamsi, A., Guo, X.-Y., Mohammadi, A., Alinejad-Rokny, H., Sejdinovic, D., Teney, D., Ranasinghe, D.C. and Abbasnejad, E. (2024). *Bayesian Low-Rank LeArning (Bella):*

A Practical Approach to Bayesian Neural Networks. arXiv:2407.20891. Available at: https://arxiv.org/abs/2407.20891.

[7] Arxiv.org. (2020). *MedBayes-Lite: Bayesian Uncertainty Quantification for Safe Clinical Decision Support*. arXiv:2511.16625. Available at: https://arxiv.org/html/2511.16625v1.

[8] Jantre, S., Bhattacharya, S. and Maiti, T. (2021). *Layer Adaptive Node Selection in Bayesian Neural Networks: Statistical Guarantees and Implementation Details*. arXiv:2108.11000. Available at: https://arxiv.org/abs/2108.11000.

[9] Wikipedia Contributors (2018). *Long short-term memory*. Wikipedia, Wikimedia Foundation. Available at: https://en.wikipedia.org/wiki/Long_short-term_memory.